

Testes de Integração e End-to-End (E2E) em APIs NestJS

Aprenda a configurar, estruturar e executar testes de integração e E2E em APIs construídas com NestJS, cobrindo tanto o uso do `TestingModule` para testes sem servidor HTTP quanto o uso de Supertest para validar fluxos completos com requisições HTTP reais.

NESTJS

TESTES E2E

Objetivos de Aprendizagem

Ao final deste material, você será capaz de dominar os principais conceitos e práticas de testes automatizados em APIs NestJS.

Diferenciar Tipos de Testes

Compreender a diferença entre testes unitários, de integração e E2E.

Configurar Ambiente E2E

Configurar o ambiente de testes E2E no NestJS corretamente.

Utilizar Supertest

Realizar chamadas HTTP automatizadas com a biblioteca Supertest.

Validar Fluxos Completos

Testar cadastro, autenticação e rotas protegidas por JWT.

Garantir Estabilidade

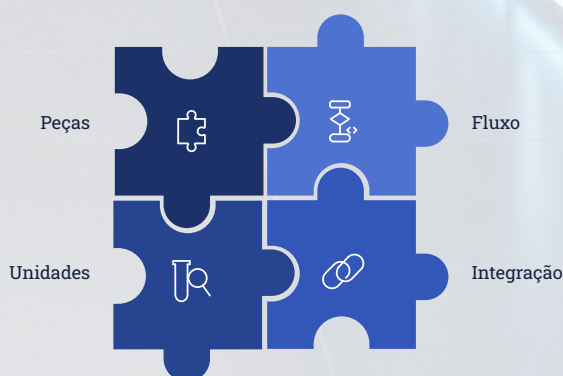
Assegurar que a API continue funcionando após evoluções futuras.

Por que Criar Testes de Integração e E2E?

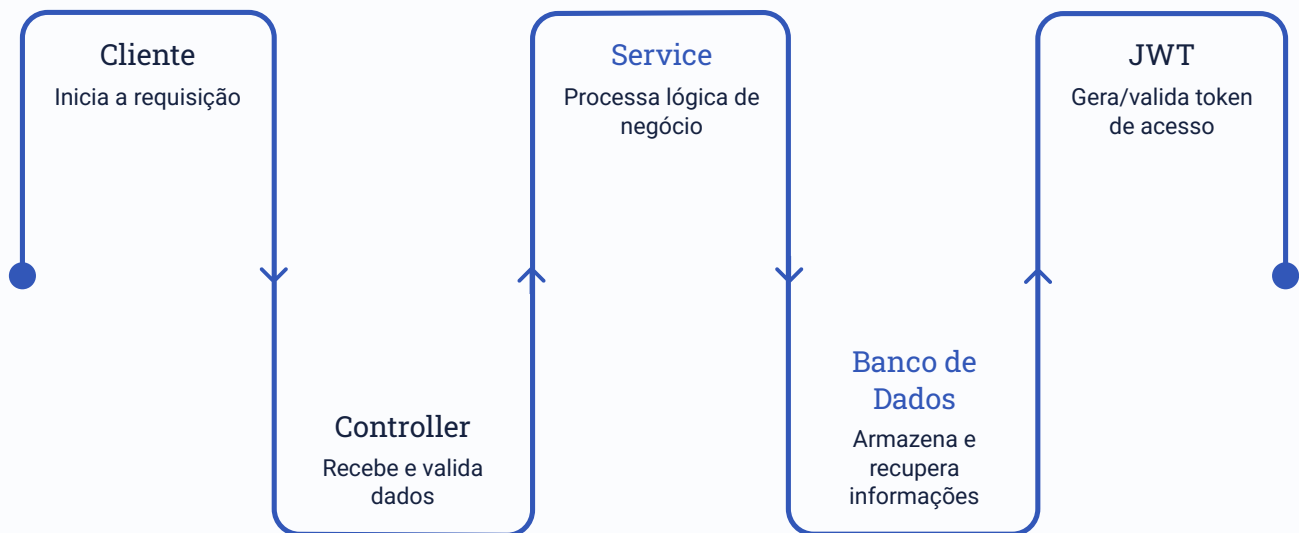
Nos testes unitários verificamos métodos isoladamente, como `AuthService.login()`, `UserService.create()` e `UserService.findByEmail()`. Entretanto, uma aplicação real envolve vários componentes trabalhando juntos.

O Problema dos Testes Unitários

Mesmo que todos os testes unitários estejam corretos, o fluxo completo pode falhar. Cada peça funciona isoladamente, mas a integração entre elas pode conter erros invisíveis nos testes unitários.



O Fluxo Real de uma Requisição



Por isso utilizamos testes de integração e E2E para validar o caminho completo.

Diferença entre Integração e E2E

Teste de Integração	Teste E2E (End-to-End)
Valida a comunicação entre módulos da aplicação. O objetivo é verificar se as partes funcionam juntas internamente.	Valida a aplicação como um usuário real . O teste realiza chamadas HTTP reais e valida as respostas completas.
<pre>AuthController AuthService UsersService</pre>	<pre>POST /auth/register POST /auth/login GET /users</pre>



Testes de Integração no NestJS – Configurando o TestingModule

Os testes de integração no NestJS utilizam o `TestingModule` para montar um módulo real da aplicação sem a necessidade de subir o servidor HTTP completo. Isso nos permite testar a comunicação entre componentes essenciais como `Controller`, `Service` e o repositório com um banco de dados real (ou em memória), mas sem a sobrecarga de realizar requisições HTTP reais. O foco é verificar se as diferentes camadas da aplicação interagem corretamente entre si no ambiente de desenvolvimento.

```
// src/auth/auth.integration.spec.ts
import { Test, TestingModule } from '@nestjs/testing';
import { AuthModule } from './auth.module';
import { AuthService } from './auth.service';
import { TypeOrmModule } from '@nestjs/typeorm';

describe('Auth - Integração', () => {
  let authService: AuthService;
  let module: TestingModule;

  beforeAll(async () => {
    module = await Test.createTestingModule({
      imports: [
        TypeOrmModule.forRoot({
          type: 'sqlite',
          database: ':memory:',
          autoLoadEntities: true,
          synchronize: true,
        }),
        AuthModule,
      ],
    }).compile();

    authService = module.get(AuthService);
  });

  afterAll(async () => {
    await module.close();
  });
});
```

Usamos SQLite em memória (`:memory:`) para que os testes de integração sejam rápidos, isolados e não dependam de um banco de dados externo.

Testes de Integração – Testando o AuthService diretamente

Nesta seção, exploraremos exemplos práticos de testes de integração, onde chamamos o `AuthService` diretamente, verificando seu comportamento real com um banco de dados em memória. Isso nos permite testar a lógica de negócios e a interação com o banco de dados sem a necessidade de uma requisição HTTP completa.

Aqui estão três exemplos de testes:

1. Registrar um usuário:

```
it('deve registrar um novo usuário', async () => {  
  const result = await authService.register({  
    name: 'João Silva',  
    email: 'joao@email.com',  
    password: '123456',  
  });  
  
  expect(result).toBeDefined();  
  expect(result.email).toBe('joao@email.com');  
});
```

2. Fazer login e retornar token:

```
it('deve autenticar e retornar access_token', async () => {  
  const result = await authService.login({  
    email: 'joao@email.com',  
    password: '123456',  
  });  
  
  expect(result.access_token).toBeDefined();  
});
```

3. Impedir e-mail duplicado:

```
it('deve lançar erro ao registrar e-mail duplicado', async () => {  
  await expect(  
    authService.register({  
      name: 'Outro',  
      email: 'joao@email.com',  
      password: 'abc123',  
    })  
  ).rejects.toThrow();  
});
```

```
    ).rejects.toThrow();  
  });
```

Nesses testes, chamamos os métodos do service diretamente — sem HTTP. O banco SQLite em memória garante isolamento total entre execuções.

Enquanto os testes de integração focam na comunicação interna entre módulos, os testes E2E simulam o comportamento real de um usuário interagindo com a API através de requisições HTTP.

O que é o Supertest e Como Configurar o Ambiente

Supertest

O Supertest é uma biblioteca utilizada para testar APIs HTTP de forma automatizada. Ele permite realizar requisições reais contra a aplicação durante os testes.

Instalação:

```
pnpm install --save-dev supertest
```

Estrutura de Arquivos

O NestJS normalmente cria a pasta `test/`. Podemos organizar os arquivos por domínio:

```
test/  
├── auth.e2e-spec.ts  
├── users.e2e-spec.ts  
└── authorization.e2e-spec.ts
```

Configurando a Aplicação para Testes

```
import { Test } from '@nestjs/testing';  
import { INestApplication } from '@nestjs/common';  
import * as request from 'supertest';  
  
describe('Auth E2E', () => {  
  
  let app: INestApplication;  
  
  beforeAll(async () => {  
    const moduleRef = await Test  
      .createTestingModule({
```

```
    imports: [AppModule],
  }).compile();

  app = moduleRef
    .createNestApplication();

  await app.init();
});
afterAll(async () => {
  await app.close();
});
});
```

O `beforeAll` inicializa a aplicação antes de todos os testes, e o `afterAll` garante que a aplicação seja encerrada corretamente após a execução.

Testando Cadastro, Login e Token JWT

Testando Cadastro de Usuário

```
it('deve cadastrar um usuário',
  async () => {

    await request(app.getHttpServer())
      .post('/auth/register')
      .send({
        name: 'Maria',
        email: 'maria@email.com',
        password: '123456'
      })
      .expect(201);
  });
```

O teste realiza uma requisição HTTP real para a API e espera o status 201 Created.

Testando Login

```
it('deve realizar login', async () => {

  const response = await request(
```

Testando Login - **continuação**

```
app.getHttpServer(),
)
.post('/auth/login')
.send({
  email: 'maria@email.com',
  password: '123456'
});
expect(response.status).toBe(200);
expect(
  response.body.access_token,
).toBeDefined();
});
```

Além do status HTTP, validamos o conteúdo retornado — o token JWT deve estar presente.

Armazenando o Token JWT

```
let accessToken: string;

// Durante o login:
accessToken = response.body.access_token;
```

Muitos testes posteriores precisarão do token. Armazená-lo em uma variável permite reutilizá-lo em outros cenários de teste.

Testando Rotas Protegidas e Cenários de Erro

Com o token JWT armazenado, podemos validar o acesso a rotas protegidas e garantir que a segurança esteja funcionando corretamente.

✓ Acessar Rota Protegida com Token

```
it('deve acessar rota protegida',
  async () => {

    await request(app.getHttpServer())
      .get('/users')
      .set(
        'Authorization',
```

✓ Acessar Rota Protegida com Token - continuação

```
`Bearer ${accessToken}`  
)  
.expect(200);  
});
```

O cabeçalho `Authorization` simula um usuário autenticado.

⊘ Bloquear Acesso sem Token

```
it('deve bloquear acesso sem token',  
  async () => {  
  
    await request(app.getHttpServer())  
      .get('/users')  
      .expect(401);  
  });
```

Esse teste garante que a segurança esteja funcionando — sem token, o acesso deve ser negado com status `401`.

✗ Rejeitar Senha Incorreta

```
it('deve rejeitar senha incorreta',  
  async () => {  
  
    await request(app.getHttpServer())  
      .post('/auth/login')  
      .send({  
        email: 'maria@email.com',  
        password: 'senha-errada'  
      })  
      .expect(401);  
  });
```

Testar cenários negativos é tão importante quanto testar cenários positivos.

⚠ Impedir E-mail Duplicado

```
it('deve impedir e-mail duplicado',
  async () => {

    await request(app.getHttpServer())
      .post('/auth/register')
      .send({
        name: 'Maria',
        email: 'maria@email.com',
        password: '123456'
      })
  })
```

Testando Autorização por Perfil e Executando os Testes

Autorização por Perfil (RBAC - Role-Based Access Control)

Suponha uma rota exclusiva para administradores: GET /users/admin. Um usuário comum autenticado deve receber status 403 Forbidden.

```
it('deve bloquear usuário comum',
  async () => {

    await request(app.getHttpServer())
      .get('/users/admin')
      .set(
        'Authorization',
        `Bearer ${accessToken}`
      )
      .expect(403);
  });
```

O teste verifica se o sistema está respeitando as permissões de perfil corretamente.

Executando os Testes

Executar todos os testes E2E:

```
pnpm run test:e2e
```

Executar um arquivo específico:

```
pnpm run test:e2e test/auth.e2e-spec.ts
```

Organize os arquivos por domínio para facilitar a execução seletiva durante o desenvolvimento.

Sempre execute os testes E2E em um ambiente de banco de dados separado do ambiente de produção para evitar contaminação de dados reais.

Estratégia Recomendada para APIs Corporativas

Fluxos mínimos recomendados para garantir cobertura adequada em APIs corporativas com autenticação e autorização.



4	4	4	12
Cenários de Autenticação	Cenários de Usuários	Cenários de Segurança	Total Mínimo Recomendado
Cadastro válido, duplicado, login válido e inválido.	Busca autenticada, sem autenticação, atualização e remoção.	Token ausente, inválido, expirado e perfil sem permissão.	Fluxos essenciais para cobertura corporativa básica.

Checklist de Validação e Conclusão

Checklist — Antes de Concluir

- ✓ Supertest instalado
- ✓ Ambiente E2E configurado
- ✓ Cadastro sendo testado
- ✓ Login sendo testado
- ✓ JWT sendo armazenado
- ✓ Rotas protegidas sendo validadas
- ✓ Cenários de erro implementados
- ✓ Cenários de autorização implementados
- ✓ Testes executando sem falhas

Conclusão

Testes unitários ajudam a validar partes isoladas da aplicação. Já os testes de integração e E2E verificam se todos os componentes realmente funcionam juntos.

Em APIs modernas, esses testes se tornam **fundamentais para garantir segurança, estabilidade e confiança** durante a evolução do sistema, permitindo que novas funcionalidades sejam adicionadas sem comprometer comportamentos já existentes.

Testes E2E são a última linha de defesa antes do usuário real — eles garantem que a API funciona de ponta a ponta, exatamente como esperado.

Parabéns! Ao dominar testes de integração e E2E com NestJS e Supertest, você garante que sua API seja robusta, segura e confiável em produção.

